

엔터프라이즈 클라우드 플랫폼 엔지니어링 포트폴리오

README 기반 독자판 · AWS · Terraform · EKS · 운영 자동화

아키텍처 설계 및 구현: 작성자 · 문서화 보조: OpenAI Codex

2026-08-10

목차

1 프로젝트 목표	2
2 범위와 증거를 읽는 방법	4
3 저장소 구조와 권장 탐색 순서	5
4 Multi-Agent Operating Model	6
5 Target Cloud Architecture	7
6 Landing Zone과 서비스 네트워크	8
7 Terraform Implementation Strategy	9
8 EKS와 Kubernetes Platform	11
9 Monitoring and Alerting	13
10 Operations Strategy	14
11 FinOps Strategy	15
12 Security and Governance	16
13 현재 구현과 검증 상태	18
14 다음 구현 단계	19
15 Definition of Done과 Production Promotion Gate	20

1 프로젝트 목표

이 프로젝트는 Terraform 리소스 몇 개를 나열하는 예제가 아닙니다. 실제 기업의 클라우드 플랫폼 팀이 함께 다뤄야 하는 아키텍처, 보안, 모니터링, 알람, 운영, 비용, 변경 승인과 Agent 협업 방식을 하나의 구축 사례로 연결한 포트폴리오입니다.

이 포트폴리오는 다음 질문에 답합니다.

- 엔터프라이즈 AWS 환경을 어떤 계정과 네트워크 구조로 설계할 것인가?
- Terraform 코드, state와 환경 경계를 어떻게 나눌 것인가?
- private EKS와 Kubernetes 플랫폼을 어떻게 구성하고 운영할 것인가?
- 보안, 접근 제어, 백업, 패치와 비용 통제를 어디에서 적용할 것인가?
- 장애와 이상 징후를 어떤 신호와 임계값으로 탐지할 것인가?
- 여러 전문 Agent를 어떤 identity, tool과 승인 경계 안에서 활용할 것인가?
- 코드와 로컬 시험, 실제 AWS 배포 증거를 어떻게 구분할 것인가?

1.1 설계 영역별 Target Scope

설계 영역	주요 범위
AWS 클라우드 및 네트워크	Organizations, Landing Zone, VPC/subnet, IPAM, TGW, routing domain
Terraform과 IaC	root/module/state ownership, 환경 분리, validation, plan과 CI/CD 승인
EKS 및 Kubernetes Platform	private EKS, node/Pod network, Istio, QoS, scaling, backup과 upgrade
Monitoring 및 Observability	CloudWatch, Prometheus/Grafana, alert policy, Mimir와 OpenTelemetry
Operations	backup/restore, scheduler, patch, CVE/EOS, artifact와 read-only 점검
Security, Governance 및 Identity	IAM/KMS, OU/SCP, WAF, Corporate IdP, Identity Center와 폐쇄망 접근
FinOps 및 Cost Governance	tag, budget, anomaly, rightsizing, network/observability 비용과 절감 검증
AI Platform 및 Multi-Agent	AI Gateway, Agent Runtime, model/tool 경계, token/cost와 evidence audit
Documentation 및 Validation	canonical 문서, schema/fixture/report 경계, 자동 검증과 PDF 시각 검증

1.2 한눈에 보는 구축 범위

AWS Organizations

- 중앙 랜딩 존: IPAM · Transit Gateway · RAM
- 서비스 네트워크: 5개 서비스 × dev/stg/prod
- 공통 환경: network · security · IAM · operations · cost
- EKS: private cluster · managed node group · Istio · Prometheus
- 운영: monitoring · backup · patch · scheduler · FinOps
- Agent: read/draft/plan/review · 사람 승인 · protected CI/CD

항목	현재 저장소에서 확인 가능한 범위
AWS 리전	서울 ap-northeast-2, 조직 정책 예외용 us-east-1
환경	dev, stg, prod
서비스	commerce, payments, analytics, customer-profile, internal-admin
서비스 네트워크	5개 서비스 × 3개 환경 = 15개 독립 VPC root
중앙 네트워크	IPAM 10.64.0.0/10, TGW ASN 64520, 4개 routing domain
공통 EKS	Kubernetes 1.35, private API, AL2023 managed node group
Terraform	19개 module 디렉터리 중 18개 구현, 28개 validation target
운영 도구	Linux 5개, EKS 1개, AWS 1개 읽기 전용 점검 스크립트
Agent Runtime	Monitoring Agent evidence collector, 고정 query, redaction, report/audit

1.3 문서가 주장하는 것과 주장하지 않는 것

구현은 저장소에 코드나 문서가 존재하고 정적 검토가 가능하다는 뜻입니다. 실제 AWS 계정에 배포되어 운영 중이라는 뜻은 아닙니다. 실제 production 완료를 주장하려면 승인된 plan, 적용 identity, ticket, 배포 후 health 와 복구 시험 증거가 별도로 필요합니다.

이 PDF는 루트 README.md의 서술 순서와 현재/목표 구분을 기준으로 작성한 독자판입니다. 세부 운영 기준은 각 canonical 문서가 소유하며, 이 문서는 전체 구조와 판단 근거를 빠르게 이해하도록 연결합니다.

2 범위와 증거를 읽는 방법

2.1 Current, Defined, Target, Evidence

같은 기능이라도 코드가 존재하는 것과 실제 운영 증거가 있는 것은 다른 완료 단계입니다.

표시	의미	예시
Current	현재 저장소에 코드, 설정 또는 실행 가능한 도구가 있음	Terraform module, read-only 점검 스크립트
Defined	정책, 계약, query 또는 승인 기준이 문서와 설정으로 정의됨	alert policy, Agent request schema
Target	목표 아키텍처이지만 아직 배포 코드나 운영 연결이 없음	중앙 AI Gateway, Mimir, Identity Center module
Evidence	로컬 시험, plan, 배포 후 확인처럼 주장에 연결할 수 있는 근거	unit test, validation log, 승인된 plan artifact

현재 구현 범위와 목표 범위를 한 표에 섞어 놓지 않는 것이 이 포트폴리오의 핵심 원칙입니다.

영역	현재 저장소에서 확인 가능	아직 목표 단계
Agent	10개 역할, request template, operator guide, Monitoring Agent MVP	사내 portal, 중앙 AI Gateway, live connector
Monitoring	11개 alert policy와 10개 Prometheus query catalog	rule 배포, 자동 severity 평가, on-call route
Terraform	Organization, Landing Zone, 서비스 VPC, 환경과 플랫폼 root	실제 계정 plan/apply와 production 운영 증거
EKS	private cluster, managed node, Istio, quota, Prometheus/Grafana	autoscaler, workload HPA/PDB, central Mimir
Identity	접근 흐름, permission set과 JIT 기준 문서화	Identity Center Terraform 배포
변경 실행	branch, patch, plan, review 계약	Agent의 production 직접 변경은 의도적으로 제외

2.2 결과를 검증하는 기준

- examples/는 synthetic fixture이며 production evidence가 아닙니다.
- schemas/는 Portal, Gateway, Runtime 사이의 machine-readable 계약입니다.
- reports/templates/는 발행 양식이고 reports/examples/는 sanitized simulation입니다.
- 실제 값이 없으면 추정하지 않고 not_available과 사유를 기록합니다.
- simulation, partial, blocked는 운영 성공이나 KPI 달성으로 합산하지 않습니다.
- 변경 결과는 request_id, trace_id, ticket, UTC window, source version과 사람의 sign-off에 연결합니다.

3 저장소 구조와 권장 탐색 순서

README는 프로젝트 전체의 단일 진입점입니다. 세부 문서는 주제별 기준을 소유하고, Terraform 디렉터리는 실제 resource와 state ownership을 보여 줍니다.

— README.md	전체 범위, 구현 상태와 포트폴리오 서사
— AGENTS.md	역할, 공통 실행 경계와 완료 기준
— agents/	운영자 가이드, 역할과 요청 template
— docs/	아키텍처, EKS, 운영, 보안, FinOps 기준
— terraform/	
— organization/	OU, SCP, Tag Policy
— landing-zone/	IPAM, TGW hub, connectivity
— services/	5개 서비스 × 3개 환경 VPC
— environments/	dev/stg/prod AWS foundation과 platform
— modules/	reusable Terraform module
— agent-runtime/	read-only Monitoring Agent MVP와 시험
— config/monitoring/	alert policy와 query catalog
— scripts/	검증, Linux/EKS/AWS 운영 점검, PDF 빌드
— schemas/	request, evidence, report 계약
— examples/	synthetic request와 evidence fixture
— reports/	월간·장애 보고서 template과 simulation
— enterprise-cloud-portfolio.pdf	

3.1 질문별 시작 위치

확인하려는 내용	먼저 볼 문서
전체 범위와 현재 상태	README.md
Agent 역할과 production 금지 경계	AGENTS.md, agents/operator-guide.md
Terraform root, module, state와 적용 순서	terraform/README.md
AWS 전체 아키텍처	docs/architecture.md
서비스 CIDR과 TGW 연결	docs/service-network-architecture.md
EKS Day-2 운영	docs/eks-operations.md
모니터링 query와 severity	docs/monitoring-alert-policy.md
운영, 비용과 보안 잔여 위험	docs/operations.md, docs/finops.md, docs/security-review.md
PDF 원고와 작성 원칙	docs/portfolio-presentation.md, docs/portfolio-outline.md

3.2 구축과 검토 순서

1. 역할, 요구사항과 전체 architecture boundary를 정의합니다.
2. Organizations, OU, SCP와 변경 승인 기준을 정합니다.
3. reusable module과 organization, landing zone, service, environment root를 구현합니다.
4. IAM, 네트워크, 암호화와 접근 제어를 검토합니다.
5. 관측성, 알람, 백업, 패치와 운영 runbook을 연결합니다.
6. 태그, 예산과 비용 이상 탐지 기준을 검토합니다.
7. CI에서 format, validation, 구조와 계약을 자동 검증합니다.
8. Reviewer가 코드와 문서의 Current/Target/Evidence 일관성을 확인합니다.
9. README를 기준으로 PDF를 생성하고 전체 페이지를 검수합니다.

4 Multi-Agent Operating Model

초기 운영 모델은 Agent에게 production을 위임하는 구조가 아니라 Human-led, Agent-assisted 방식입니다. Agent는 승인된 정보 조회, 문서·patch·plan·review artifact 작성과 검증을 지원하고, 운영자가 근거를 확인해 최종 판단합니다.

4.1 역할과 산출물

Agent	책임	주요 산출물
Architecture	요구사항과 module boundary	architecture decision, target architecture
Terraform	코드, module, root와 state 분리	Terraform patch, plan 요약
Governance	Organizations, OU, SCP, 승인	governance model, compliance checklist
Security	IAM, 네트워크, 암호화와 정책	security baseline, risk review
Monitoring	metric, log, alert와 장애 증적	alert policy, incident report
Operations	backup, patch, CVE/EOS, lifecycle	runbook, monthly platform report
FinOps	tag, budget, 사용량과 절감안	cost policy, optimization report
CI/CD	검증, plan review와 배포 gate	pipeline, approval workflow
Reviewer	코드, 보안과 운영 위험 독립 검토	review report, backlog
Documentation	README, 세부 문서와 PDF	portfolio narrative, report template

4.2 표준 실행 흐름

Operator request

- Corporate identity와 요청 계약 검증
- Agent profile, model/tool, token/cost quota 적용
- short-lived credential로 read/draft/plan/review
- evidence, patch, plan 또는 review artifact 생성
- 사람과 Reviewer 검토
- protected CI/CD가 승인된 변경만 배포
- request_id와 trace_id로 감사 연결

Agent mode는 명시적으로 제한됩니다.

Mode	허용	금지
read	repository, metric, log, inventory 분석	파일과 cloud 변경
draft	문서, code patch, runbook 초안	merge, deploy, cloud API 변경
plan	fmt, validate, plan, 영향과 rollback 분석	apply, restart, delete, purchase
review	code, plan, policy와 evidence 독립 검토	원안 자동 승인과 deploy

apply는 Agent mode가 아닙니다. prod 배포는 plan artifact, ticket과 지정 승인자를 확인하는 protected CI/CD deployment role만 수행합니다.

4.3 현재 실행 가능한 Agent 범위

현재 구현은 Monitoring Agent의 read-only incident evidence collector입니다. 고정 query catalog, URL/DNS와 runtime identity 검증, secret/PII redaction, 보고서와 audit record 생성을 포함합니다. Corporate IdP, 중앙 AI Gateway, portal과 live connector는 Target architecture입니다.

5 Target Cloud Architecture

AWS workload VPC는 인터넷 경계를 직접 갖지 않는 private spoke 구조로 설계합니다. 외부·사내 ingress와 egress는 Landing Zone의 중앙 연결 계층을 통과하고, workload는 필요한 AWS 서비스에 VPC endpoint로 접근합니다.

5.1 전체 계층

Corporate IdP / Operator / CI



AWS Organizations · OU · SCP · Tag Policy



Landing Zone: IPAM · Transit Gateway · RAM · central ingress/egress



Workload VPC: LB · AP · DB · Node · Pod · TGW · EKS Cluster subnet



Private EKS: managed node · Pod Identity · Istio · Prometheus/Grafana



CloudWatch · Backup · Patch · Inspector · Budget · Cost Anomaly

5.2 환경별 운영 의도

환경	목적	운영 기준
dev	빠른 개발과 module 검증	낮은 비용, 축소된 용량, scheduler 사용
stg	production 전 통합 검증	prod와 유사한 네트워크·알람, 변경 rehearsal
prod	실제 운영을 위한 강화 기준	3 AZ, 보존 확대, Vault Lock, 수동 승인

5.3 Workforce Identity와 AI Platform

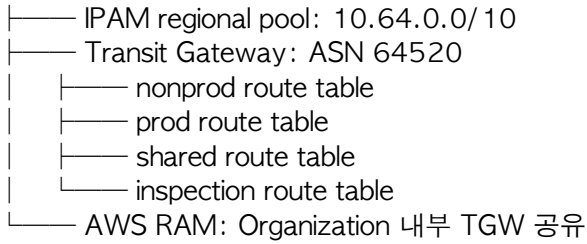
- Corporate IdP를 SAML 2.0과 SCIM으로 IAM Identity Center에 연결합니다.
- IdP group을 permission set과 account에 연결하고 IAM user와 장기 access key를 피합니다.
- prod 권한에는 MFA, JIT group, 짧은 session과 ticket을 요구합니다.
- private AI Gateway는 identity, model/tool allowlist, data classification과 token/cost quota를 검증합니다.
- Agent는 실행 시점에만 범위가 제한된 credential을 받고 production API를 직접 변경하지 않습니다.
- token, latency, error, policy result와 estimated cost는 중앙 usage lake와 dashboard에 기록합니다.

Identity Center와 AI Gateway는 현재 문서화된 목표 구조입니다. 현재 Terraform 배포 범위와 혼동하지 않습니다.

6 Landing Zone과 서비스 네트워크

6.1 중앙 네트워크

Network Account



Service VPC attachment → 환경별 association과 허용 route

TGW의 기본 association과 propagation은 비활성화되어 있습니다. dev와 stg는 nonprod, prod는 prod routing domain에 연결하고, 서비스 간 route는 connectivity root의 명시적 허용 목록으로만 생성합니다.

6.2 공통 환경 VPC

환경	VPC	AZ	기본 경로	Interface Endpoint	Backup
dev	10.10.0.0/16	2	Landing Zone TGW	없음	14일
stg	10.15.0.0/16	2	Landing Zone TGW	ECR, Logs, SSM 5종	35일
prod	10.20.0.0/16	3	Landing Zone TGW	EC2, ECR, Logs, SSM, STS 등 8종	35일 + Vault Lock

6.3 서비스 VPC의 subnet 분리

서비스 VPC / AZ

├── LB subnet	내부 ALB/NLB ENI
├── AP subnet	일반 애플리케이션 ENI
├── DB subnet	기업 CIDR 외 기본 route 없음
├── EKS Node subnet	Managed Node Group primary ENI
├── VPC CNI Pod subnet	Pod secondary ENI
├── TGW subnet / 28	Transit Gateway attachment
└── EKS Cluster subnet / 28	control plane x-ENI

Workload VPC에는 Public Subnet, Internet Gateway와 NAT Gateway가 없습니다. LB/AP/Node/Pod/EKS Cluster의 기본 route는 TGW로 전달하고, DB subnet은 기업 CIDR만 전달합니다.

6.4 5개 서비스와 15개 VPC

서비스	workload	dev	stg	prod
commerce	대규모 EKS/API	10.64.0.0/20	10.64.32.0/19	10.65.0.0/16
payments	transaction	10.72.0.0/22	10.72.8.0/21	10.73.0.0/18
analytics	batch/EKS	10.76.0.0/20	10.76.64.0/18	10.77.0.0/16
customer-profile	API/data	10.80.0.0/22	10.80.8.0/21	10.81.0.0/19
internal-admin	내부 업무	10.84.0.0/22	10.84.4.0/22	10.84.16.0/20

서비스 root는 VPC, 7개 subnet tier, route table과 TGW attachment를 생성합니다. 서비스별 application, EKS cluster, ALB와 RDS는 현재 범위에 포함되지 않습니다.

7 Terraform Implementation Strategy

Terraform은 조직 정책, 중앙 네트워크, 서비스 네트워크, 공통 AWS foundation과 Kubernetes platform을 서로 다른 state로 분리합니다. 분리 기준은 폴더 모양이 아니라 blast radius, 권한과 lifecycle입니다.

7.1 Root와 state ownership

terraform/	
├── organization/	Organization · OU · SCP · Tag Policy
├── landing-zone/	
│ ├── ipam/	IPAM pool
│ ├── network-hub/	TGW · RAM · routing domain
│ └── connectivity/	association · 허용 route
├── services/ {service}/ {env}/	15개 서비스 VPC state
├── environments/ {env}/	AWS 공통 foundation state
│ └── platform/	Kubernetes · Helm state
└── modules/	재사용 module

State	생성 범위	분리 이유
organization	Organizations, OU, SCP, Tag Policy	조직 전체 정책을 workload 배포와 격리
landing-zone/ipam	enterprise와 서비스 address pool	CIDR 수명주기와 할당 책임 분리
landing-zone/network-hub	TGW, RAM, 4개 route table	중앙 네트워크 권한 분리
services/*/*	private VPC, 7개 subnet tier, attachment	서비스·환경별 독립 변경
landing-zone/connectivity	association과 허용 route	연결 정책을 VPC 생성과 분리
environments/ {env}	VPC, IAM, security, operations, cost, EKS	AWS resource lifecycle 관리
environments/ {env}/platform	Istio, namespace policy, Prometheus/ Grafana	Kubernetes API와 chart lifecycle 분리

7.2 Reusable module

구분	주요 module
조직	organization, scp-policy
네트워크	network, service-vpc, transit-gateway-hub, transit-gateway-routing
보안·접근	security, iam, security-group, route-policy, waf
플랫폼	workload-environment, eks, kubernetes-platform
운영·관측	observability, monitoring-agent-access, operations, cost

Root module은 provider, backend, locals와 module 호출만 담당합니다. 실제 resource는 terraform/modules/*에 두고, 환경 차이는 variable로 표현합니다. 중요한 output은 다음 state가 소비할 수 있도록 명시적으로 노출합니다.

7.3 적용과 변경 순서

organization

- landing-zone/ipam
- landing-zone/network-hub
- service VPC
- landing-zone/connectivity
- environment foundation
- Kubernetes platform

동일한 security-group rule, route destination 또는 Kubernetes object를 두 state가 동시에 관리하지 않습니다. 변경 빈도가 높다는 이유만으로 state를 나누지 않고, 소유 팀, 승인 권한 또는 lifecycle이 다를 때 분리합니다.

모든 Agent는 prod에서 직접 terraform apply하지 않습니다. Agent는 format, validate, plan과 영향 분석까지만 지원하고, protected CI/CD가 plan hash, ticket과 approver를 확인한 뒤 적용합니다.

8 EKS와 Kubernetes Platform

8.1 Private EKS foundation

Private EKS 1.35

- └── EKS Cluster subnet: control plane x-ENI
- └── Node subnet: managed node group primary ENI
- └── Pod subnet: VPC CNI secondary ENI
- └── Access Entry: cluster admin · Monitoring Agent
- └── KMS: Kubernetes Secret · node EBS · CloudWatch Logs
- └── Managed Add-on 6종
- └── Managed Node Group
- └── Kubernetes Platform State
 - └── Istio
 - └── application namespace policy
 - └── Prometheus · Grafana · Alertmanager

항목	현재 구성
API endpoint	private access 사용, public access 비활성
인증	API_AND_CONFIG_MAP, bootstrap creator admin 비활성
관리자	EKS Access Entry + AmazonEKSClusterAdminPolicy
Secret 암호화	EKS 전용 KMS key와 rotation
Node OS	EKS AL2023 managed node group
Node volume	암호화 gp3, IMDSv2 필수, detailed monitoring
Add-on	VPC CNI, CoreDNS, kube-proxy, Pod Identity Agent, EBS CSI, CloudWatch Observability
VPC CNI	custom networking, prefix delegation, AZ별 ENIConfig

8.2 환경별 Managed Node Group

환경·그룹	Instance	Capacity	min / desired / max	Disk
dev general	t3.large	Spot	1 / 1 / 3	30Gi
stg general	m6i.large	On-Demand	2 / 2 / 5	50Gi
prod system	m7i.large	On-Demand	3 / 3 / 6	80Gi
prod application	m7i.large, m6i.large	On-Demand	3 / 3 / 12	80Gi

prod system group은 CriticalAddonsOnly=true:NoSchedule taint를 사용해 application node와 분리합니다.

8.3 Kubernetes policy와 현재 경계

구성	현재 내용
Service Mesh	revision 기반 Istio, namespace injection, STRICT mTLS
Namespace	application-dev, application-stg, application-prod
자원 정책	환경별 ResourceQuota와 container LimitRange
PriorityClass	platform-critical, application-high, batch-low
Metrics	kube-prometheus-stack, Grafana, Alertmanager
PDB	module interface 존재, 실제 workload resource 0개
Autoscaling	Karpenter, Cluster Autoscaler, workload HPA/KEDA 미구현
NetworkPolicy	미구현
AWS Load Balancer Controller	미설치

desired_size drift 무시하는 자동 확장 controller가 있다는 뜻이 아닙니다. Probe, topology spread, PDB와 autoscaling은 실제 workload와 함께 rehearsal해야 production readiness를 주장할 수 있습니다.

9 Monitoring and Alerting

관측성은 metric, log, trace, alert와 dashboard를 역할별로 분리합니다. CloudWatch는 AWS native signal의 기본 계층이고, Prometheus는 Kubernetes local scrape와 빠른 제어 loop, Grafana는 통합 조회 계층입니다.

9.1 신호 수집 흐름

VPC Flow Logs → CloudWatch Logs → Metric Filter → Alarm
 EKS control plane 5종 → CloudWatch Logs → SNS
 Container Insights 4종 → CloudWatch Logs → Email

Node · Pod · Istio metric → Prometheus → Grafana
 ↳ Alertmanager

Monitoring Agent → Logs Insights · EKS Describe · Kubernetes read-only

Signal	dev / stg / prod 보존	현재 저장 위치
VPC Flow Logs	90 / 90 / 365일	KMS 암호화 CloudWatch Logs
EKS control plane	90 / 90 / 365일	전용 KMS CloudWatch Logs
Container log 4종	30 / 90 / 365일	전용 KMS CloudWatch Logs
Kubernetes metric	7 / 15 / 30일	Prometheus 50Gi PVC
장기 metric	목표 30 / 90 / 400일	Mimir는 Target architecture

9.2 Alert policy

지표	Warning	Critical
Host CPU	80% 이상 10분	90% 이상 5분
Host memory	80% 이상 10분	90% 이상 5분 또는 OOM
Disk-inode	80% 이상 15분	90% 이상 10분
Pod CPU request	80% 이상 10분	95% 이상 5분
Container memory limit	80% 이상 10분	90% 이상 5분 또는 OOMKilled
Target 5xx	2% 이상 5분	5% 이상 5분
Node NotReady	1개 이상 2분	1개 이상 5분

기계 판독 가능한 alert policy는 query ID, 지속 시간, Warning/Critical, missing-data와 복구 조건을 검증합니다. 현재 AWS 알람은 VPC rejected flow filter와 SNS 흐름이 구현되어 있습니다. Prometheus와 Grafana는 설치되지만 Alertmanager receiver/route, 중앙 immutable log archive와 Mimir 장기 저장은 아직 구현되지 않았습니다.

9.3 운영자가 확인할 관점

- CPU 한 지표만 보지 않고 load, run queue, throttling과 saturation을 함께 봅니다.
- memory는 available, working set, OOM과 eviction을 함께 봅니다.
- disk는 용량, inode, latency와 I/O queue를 분리합니다.
- application alarm은 최소 traffic과 error ratio를 함께 사용해 저트래픽 오탐을 줄입니다.
- NoData는 정상으로 단정하지 않고 수집기 장애와 workload 부재를 구분합니다.

10 Operations Strategy

운영 자동화는 백업, scheduler, patch, CVE/EOS, OS lifecycle과 정기 보고를 포함합니다. 자동화가 production을 임의로 바꾸지 않도록 tag, 환경과 승인 조건을 함께 둡니다.

10.1 Tagging

Tag	목적	예시
Environment	환경과 비용 분리	dev, stg, prod
Owner	책임 팀	platform-team
Service	업무 서비스	commerce, payments
CostCenter	비용 귀속	cloud-platform
ManagedBy	관리 주체	terraform
Backup	백업 선택	daily, critical
Schedule	실행 시간	office-hours

10.2 Backup과 복구

- dev는 14일, stg와 prod는 35일 기본 보존을 적용합니다.
- prod는 선택형 Vault Lock으로 삭제 방지 기간을 강화합니다.
- backup 성공만으로 복구 가능성을 주장하지 않고 restore drill과 RTO/RPO 측정이 필요합니다.
- EKS는 cluster resource와 persistent data를 나누어 backup/restore하고, 격리 환경에서 namespace와 data 복구를 검증합니다.

10.3 Scheduler

- dev와 stg의 Environment와 Schedule=office-hours tag가 모두 일치하는 EC2/RDS만 조회합니다.
- 기본값은 dry-run이며 prod 활성화는 Terraform precondition과 Lambda runtime에서 이중 차단합니다.
- EKS node group은 EC2/RDS scheduler 대상에서 제외합니다.

10.4 Patch, CVE와 EOS

- Systems Manager Patch Manager baseline과 maintenance window를 환경별로 분리합니다.
- Inspector v2로 EC2, ECR과 Lambda 취약점을 탐지합니다.
- critical CVE, package repository, image provenance와 OS EOS를 정기 검토합니다.
- production patch는 backup, health check, rollback owner와 maintenance window를 연결합니다.

10.5 읽기 전용 운영 도구

현재 저장소는 Linux host/resource/network/patch 점검 5개, EKS health 1개, CloudWatch log retention audit 1개 스크립트를 제공합니다. 이 도구는 조회와 보고만 수행하며 package 설치, restart, kubectl apply/delete/patch와 AWS 변경 API를 실행하지 않습니다.

11 FinOps Strategy

FinOps는 월말 청구서 확인이 아니라 설계, tag, 예산, anomaly, scheduler와 검증된 절감 효과를 하나의 흐름으로 관리합니다.

11.1 환경별 비용 통제

환경	월 예산	Cost Anomaly	Scheduler	네트워크 구성
dev	USD 300	USD 50 이상	평일 EC2/RDS, 기본 dry-run	TGW + S3 Gateway Endpoint
stg	USD 1,000	USD 100 이상	평일 EC2/RDS, 기본 dry-run	TGW + Interface Endpoint 5종
prod	USD 5,000	USD 300 이상	사용하지 않음	TGW + Interface Endpoint 8종

알림	기준	채널
Forecasted	예산 50% 초과 예측	KMS 암호화 SNS
Actual Warning	실제 비용 80% 초과	KMS 암호화 SNS
Actual Critical	실제 비용 100% 초과	KMS 암호화 SNS
Cost Anomaly	서비스별 일간 절대 영향 임계값 초과	KMS 암호화 SNS

11.2 비용 최적화 판단

- 비용은 Environment, Service, Owner, CostCenter tag로 귀속합니다.
- NAT Gateway 대신 중앙 TGW 경로를 선택했지만 TGW data processing과 중앙 egress 비용을 함께 관찰합니다.
- VPC endpoint는 보안과 전송 비용을 함께 검토해 환경별로 수를 다르게 둡니다.
- idle load balancer, unattached EBS, unused EIP와 과대 instance는 정기 보고 대상으로 관리합니다.
- 절감액은 baseline, 적용 기간, 서비스 품질과 청구 데이터가 연결된 경우에만 실적으로 기록합니다.

현재 저장소에는 Budget, Cost Anomaly와 scheduler 정책이 구성되어 있지만 실제 청구 데이터와 실현 절감액은 포함되어 있지 않습니다.

12 Security and Governance

보안은 배포 후 추가하는 기능이 아니라 Terraform의 기본값과 변경 절차에 포함합니다.

12.1 보안 계층

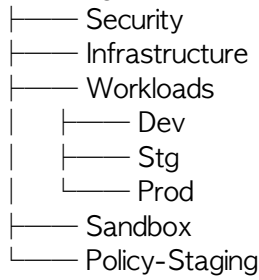
Organization SCP

- IAM deployment / audit / break-glass role
- private VPC · endpoint · flow log
- EKS private API · Access Entry · Pod Identity
- KMS Secret · EBS · CloudWatch Logs 암호화
- GuardDuty · Security Hub · Inspector
- WAF와 중앙 inspection Target

Boundary	현재 구현된 통제
Organization	nested OU, deny-leave, audit protection, region deny, tag policy
Identity	explicit trust, GitHub OIDC subject allowlist, audit role, MFA break-glass interface
Data	rotating KMS, EBS default encryption, EKS Secret/volume/log encryption
Network	private EKS, subnet 분리, TGW 중앙 경로, DB route isolation, endpoint
Runtime	Access Entry, AL2023, IMDSv2, Pod Identity, strict mTLS
Detection	GuardDuty, Security Hub, Inspector, Flow Logs, CloudWatch/SNS
Edge	WAF managed rule, rate limit, log와 sensitive-header redaction module

12.2 Organization과 정책

AWS Organizations Root



정책	연결 대상	통제
DenyLeaveOrganization	Workloads, Infrastructure, Security, Sandbox	조직 탈퇴 차단
DenyDisableAuditServices	Workloads	CloudTrail, Config, GuardDuty, Security Hub 보호
DenyUnapprovedRegions	Workloads	승인 리전 외 workload API 제한
DenyDeletePublicAccessControls	Workloads	S3 public access control 삭제 제한
EnterpriseTagPolicy	Workloads, Sandbox	Environment와 ManagedBy 표준화

Organization, OU와 policy attachment 코드는 존재하지만 aws_organizations_account와 Control Tower account vending은 현재 범위가 아닙니다. SCP는 Policy-Staging OU에서 account 단위로 검증하고 break-glass rehearsal 후 확대해야 합니다.

12.3 Production 전 잔여 위험

- Organization CloudTrail, Config aggregator와 immutable central log archive 구현
- private EKS에 접근할 VPN, Direct Connect 또는 self-hosted runner 확보
- Grafana bootstrap secret을 Secrets Manager와 External Secrets로 전환
- WAF를 실제 ingress에 count로 연결한 뒤 false positive 검토와 block 승격
- inspection VPC와 AWS Network Firewall/UTM route 구현
- sandbox에서 plan, apply, restore와 destroy까지 실행한 runtime evidence 확보

13 현재 구현과 검증 상태

13.1 현재 저장소 산출물

- 10개 Agent 역할, 역할별 request template, operator guide와 adoption scenario
- Monitoring Agent request, evidence, redaction, report와 audit 계약
- Organizations, Landing Zone, 서비스 VPC 15개, 공통 환경과 platform root
- Terraform module 디렉터리 19개 중 18개 구현
- private EKS, AL2023 node, managed add-on, Pod Identity와 KMS
- Istio, ResourceQuota, LimitRange, PriorityClass, Prometheus/Grafana
- backup, Vault Lock, scheduler, SSM patch baseline, Inspector
- Budget, Cost Anomaly, SNS 50/80/100 percent 알림
- Linux, EKS와 AWS 읽기 전용 운영 점검 7개
- alert policy, schema, fixture, 보고서 template과 sanitized simulation
- GitHub Actions와 로컬 validation script

13.2 자동 검증의 의미

검증	현재 결과	증명 범위
Agent Runtime unit test	25개 통과	request/security/read-only/report 계약
Terraform 대상	28개 target 정의	root/module 구조와 검증 범위
Terraform 최신 로컬 실행	일부 정적 검사 통과, provider handshake에서 중단	28/28 validate 성공을 주장하지 않음
Monitoring policy	JSON과 query catalog 형식 검사	query ID, threshold, M/N 계약
Report library	template/example/schema smoke 검증	산출물 역할과 필수 field
PDF	text, 목차, bookmark, 전체 페이지 render 검사	문서 구조와 시각 품질

로컬 unit test와 synthetic fixture는 실제 AWS 장애 대응 정확도, MTTR 개선, 비용 절감 또는 production readiness를 증명하지 않습니다. 운영 성과는 live baseline, 실제 source, operator correction과 accountable sign-off가 있어야 합니다.

14 다음 구현 단계

현재 코드와 목표 아키텍처 사이의 차이는 다음 순서로 줄입니다.

1. Sandbox AWS organization/account에서 승인된 plan, apply, restore와 destroy 증적을 생성합니다.
2. Security/Log Archive account ID를 확정하고 Organization CloudTrail과 Config aggregator를 구현합니다.
3. EKS ingress와 AWS Load Balancer Controller를 구성한 뒤 WAF를 count에서 block으로 단계 승격합니다.
4. 현재 Prometheus cardinality와 비용 baseline을 측정하고 dev Mimir/S3/KMS/auth gateway PoC를 진행합니다.
5. 한 application에 OpenTelemetry SDK/Collector를 적용하고 drop/retry와 PII 음성 시험을 수행합니다.
6. Prometheus Adapter allowlist, 필요한 queue workload의 KEDA와 autoscaler를 dev/stg에서 검증합니다.
7. workload HPA, PDB, topology spread, NetworkPolicy와 AuthorizationPolicy를 구현합니다.
8. Identity Center permission set과 account assignment를 Terraform module로 구현합니다.
9. private AI Gateway, Agent Runtime, Bedrock endpoint와 usage dashboard를 단계적으로 구현합니다.
10. Trivy, Checkov와 OPA 정책 검사를 protected CI release gate에 추가합니다.
11. 운영 KPI baseline과 actual을 원본 evidence와 연결하고 월간 scorecard를 발행합니다.
12. README의 Current/Target/Evidence와 실제 AWS 결과를 기준으로 이 PDF를 계속 갱신합니다.

14.1 우선순위 판단 기준

- production blast radius를 줄이는 통제를 먼저 구현합니다.
- 실제 운영 증거가 없는 자동화보다 plan, restore와 observability evidence를 우선합니다.
- Agent 전체를 한 번에 승격하지 않고 use case와 environment 조합별로 maturity를 관리합니다.
- target architecture를 구현 완료나 비용 절감 실적으로 표현하지 않습니다.

15 Definition of Done과 Production Promotion Gate

15.1 저장소 산출물의 완료 기준

- Terraform 구조가 환경과 state lifecycle별로 분리되어 있습니다.
- module input/output과 ownership이 명확합니다.
- Organizations, OU와 SCP 기반 governance 구조가 있습니다.
- 보안, 모니터링, 운영과 FinOps 책임이 분리되어 있습니다.
- EKS logging, QoS, 가용성, backup, upgrade와 incident 기준이 설명되어 있습니다.
- Prometheus, Mimir, OpenTelemetry, Adapter/KEDA의 Current와 Target 경계가 설명되어 있습니다.
- read-only 운영 점검과 자동 계약 검증이 있습니다.
- Agent command, tool permission과 production 승인 경계가 정의되어 있습니다.
- schema, fixture, template, simulation과 실제 evidence의 역할이 분리되어 있습니다.
- 월간·장애 보고서가 사실, 가설, gap, action, approval과 evidence를 구분합니다.
- README를 기준으로 PDF를 재생성하고 전체 페이지를 검수할 수 있습니다.

15.2 Production Promotion Gate

저장소의 Definition of Done을 만족해도 다음 조건 없이는 production 완료로 승격하지 않습니다.

- 실제 account와 region에 대한 승인된 terraform plan과 plan hash
- 적용 identity, change ticket, approver, maintenance window와 rollback owner
- 배포 후 health, log, metric, backup과 비용 검증 evidence
- EKS restore, upgrade, drain/PDB와 autoscaling의 격리 환경 rehearsal
- Monitoring query와 Warning/Critical rule의 synthetic alarm과 missing-data 시험
- Agent 결과와 원본 query/plan의 표본 대조, 운영자 수정·반려 이력
- Target이나 fixture 결과를 실제 배포·절감·MTTR 성과로 표현하지 않았다는 검토

15.3 마무리

이 포트폴리오의 핵심은 리소스 수가 아니라 경계의 명확성입니다. 조직 정책과 workload state, AWS foundation과 Kubernetes platform, Agent 제안과 production 실행, 정의된 기준과 실제 증거를 분리해 설계했습니다. 그 위에서 Terraform, EKS, 관측성, 운영 자동화와 FinOps를 하나의 엔터프라이즈 플랫폼 이야기로 연결합니다.